

Configuring Profiles

Profiles are an advanced feature that allow for grouping and filtering testruns. A profile may be set using the `--profile` argument or by inputting a value in the advanced options from the web ui.

It may be useful to see a list of existing profiles as options in the form. If you have such a list, you may set it in your locustfile:

```
from locust import events

@events.init.add_listener
def on_locust_init(environment, **kwargs):
    environment.web_ui.template_args["all_profiles"] = ["one", "two", "three"]
```

=====

For custom arguments: <https://docs.locust.io/en/stable/extending-locust.html#custom-arguments>

=====

Customization of statistics settings

Default configuration for Locust statistics is set in constants of stats.py file. It can be tuned to specific requirements by overriding these values. To do this, import locust.stats module and override required settings:

```
import locust.stats
locust.stats.CONSOLE_STATS_INTERVAL_SEC = 15
```

It can be done directly in Locust file or extracted to separate file for common usage by all Locust files.

The list of statistics parameters that can be modified is:

Parameter name	Purpose
STATS_NAME_WIDTH	Width of column for request name in console o
STATS_TYPE_WIDTH	Width of column for request type in console ou
CSV_STATS_INTERVAL_SEC	Interval for how frequently the CSV file is writte
CONSOLE_STATS_INTERVAL_SEC	Interval for how frequently results are written to
HISTORY_STATS_INTERVAL_SEC	Interval for how frequently results are written to
CURRENT_RESPONSE_TIME_PERCENTILE_WINDOW	Window size/resolution - in seconds - when cal
PERCENTILES_TO_REPORT	List of response time percentiles to be calculate
PERCENTILES_TO_STATISTICS	List of response time percentiles in the screen c
PERCENTILES_TO_CHART	List of response time percentiles in the screen c



=====

Customizing response time bucketing

Response times are grouped into histogram buckets before being stored in the `response_times` dict. By default, Locust rounds to approximately 2 significant digits (e.g. 147 becomes 150, 3432 becomes 3400). This keeps the dict small, which matters in distributed mode where the dict is serialized from workers to master.

You can replace the bucketing function to change this behaviour:

```
import locust.stats
from math import floor, log10

def my_bucket_function(response_time: int | float) -> int:
    """Example: bucket to 3 significant figures."""
    if response_time == 0:
        return 0
    return int(round(response_time, -int(floor(log10(abs(response_time)))) + 2))

locust.stats.bucket_response_time = my_bucket_function
```

The replacement function receives a single numeric argument (the response time in milliseconds) and must return a numeric value to use as the dict key. Keep in mind that more unique keys means more data transferred in distributed mode.

Parameter name	Purpose
locust.runners.WORKER_LOG_REPORT_INTERVAL	Interval for how frequently worker logs are reported to the master. Can be disabled by setting to a negative number
locust.web.HOST_IS_REQUIRED	Makes the host field for the webui required

Load generation performance

If Locust prints a warning about high CPU usage (`WARNING/root: CPU usage above 90%! ...`) try the following:

- Run Locust [distributed](#) to utilize multiple cores & multiple machines
- Try switching to [FastHttpUser](#) to reduce CPU usage
- Check to see that there are no strange/infinite loops in your code

Also, if you are using a custom client (not `HttpUser` or `FastHttpUser`), make sure any client library you are using is [event-friendly](#) otherwise it will block the entire Python process (essentially limiting you to one user per worker)

If you're doing really high throughput or using a lot of bandwidth, you may also want to check out your network utilization and other OS level metrics.

What are open and closed workload models?

Open and closed workload models are two different ways to define the load on your system.

In an open workload, requests are **sent at a certain rate**, regardless of the system's response times or the number of requests currently being processed.

In a closed workload, **only a certain number of concurrent users are running**, and the request rate is implicitly limited by the response times of your system, because a user can't trigger a new request until the previous one has completed.

The primary result/outcome variable of the closed workload test is achieved throughput, typically measured in requests or transactions per second.

```
class ClosedWorkload(HttpUser):

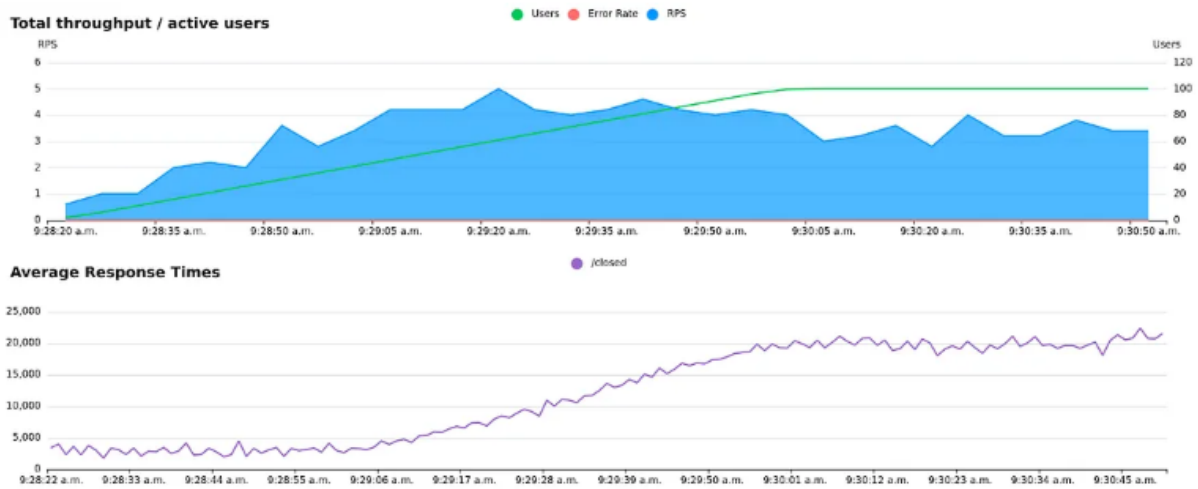
    wait_time = constant(10) # sleep 10s after each task, regardless of execution time

    @task

    def my_task(self):

        ...

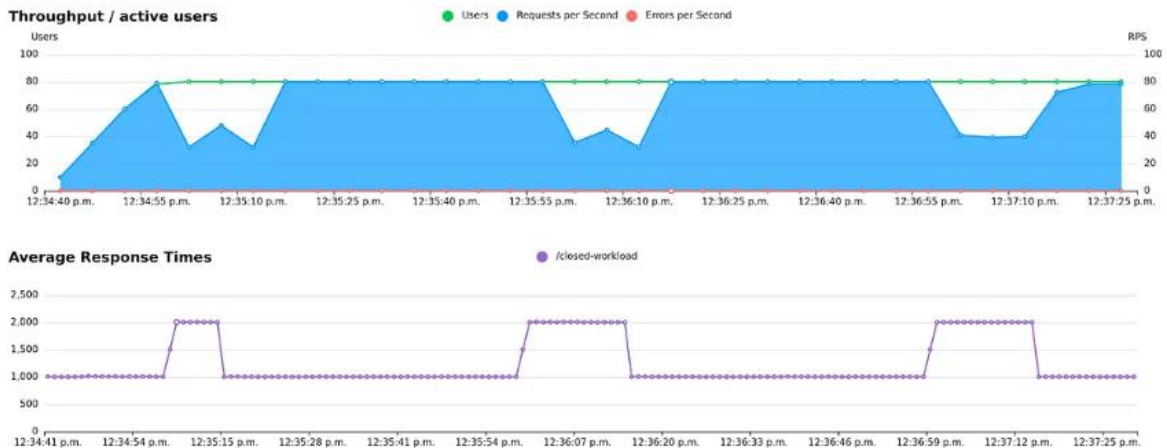
        *****
```



Ramp up using closed load. Notice how as new users are added, the response times start increasing and throughput starts to flatten, but the system is not completely overwhelmed. (Graphs are from locust.cloud)



As the test continues, it is clear that this system can't really handle more than around 4 requests/s, even if we were to accept these very long response times.



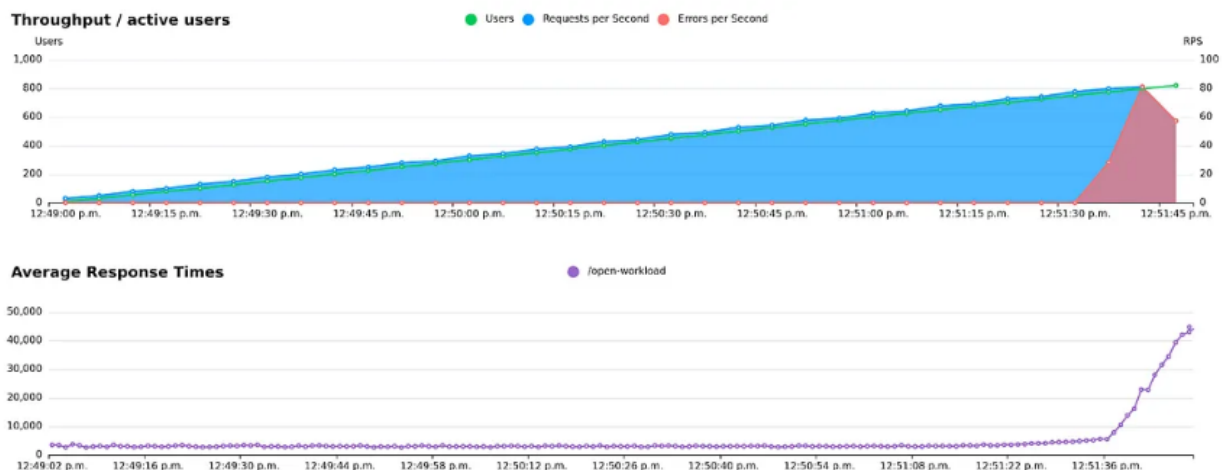
Closed load: Variations in response times cause variations in throughput ←

Open workload is the honey badger of load tests — it doesn't care about response times and keeps going at the pace you told it to. **In an open load test, you might target 80 transactions per second**, with an uncapped or **very large number of concurrent users**. It is sometimes referred to as the *arrival rate*.

The **primary result/outcome** variable of a load test using an open workload is response times, **typically measured in milliseconds or seconds**.

Most load testing tools support a type of adaptive sleep that allows you to simulate open load (constant_pacing in Locust, Constant Throughput timer in JMeter). Some tools also support open load outright, by launching additional users/threads if throughput drops below a target level. It works by filling out the time between iterations/requests so that requests are sent at a stable pace, regardless of response times:

```
class OpenWorkload(HttpUser):
    wait_time = constant_pacing(10) # max 1 iteration per 10 seconds
    ...
```



800+ users doing 1 request per 10 seconds. Around 70 RPS you can see the response times start to increase, but because there is no limit on the throughput we quickly overload the system, and we start to get error responses.

So, when is open load testing useful?

If you want to do repeated runs to understand resource utilization, then **you should use open load so you can compare identical runs with different versions of the system under test**. A closed load will be impacted by your system's response times, whereas with an open load, the load is (theoretically) stable regardless of response times.

Sometimes you need to give some thought to how your users might behave — are they a limited group of people who might “naturally” have a closed behaviour, because they won't request the next page/API call until the previous one finished? Or are your client applications making requests at the same rate regardless of response times? Or maybe your users are impatient and will start refreshing the page once response times exceed a certain threshold, *increasing* the load as response times deteriorate?

A single process running **Locust can simulate a reasonably high throughput**. For a simple test plan and small payloads, **it can make more than a thousand requests per second**, possibly over ten thousand if you use [FastHttpUser](#).

To simplify startup, you can use the `--processes` flag. It will launch a master process and the specified number of worker processes. It can also be used in combination with `--worker`, then it will only launch workers. This feature relies on `fork()` so it doesn't work on Windows. 

Note

Because Python cannot fully utilize more than one core per process (see [GIL](#)), you need to run one worker instance per processor core in order to have access to all your computing power.

Note

There is almost no limit to how many Users you can run per worker. Locust/gevent can run thousands or even tens of thousands of Users per process just fine, as long as their total request rate (RPS) is not too high.

If Locust is getting close to running out of CPU resources, it will log a warning. If there is no warning but you are still unable to generate the expected load, then the problem must be [Increasing the request rate](#).

```
(.env) C:\Tests\j-l-qa-testing\Python-Locust\simple_test>locust -f . --processes 4
[2026-05-06 16:45:47,546] Jafar/INFO/locust.main: Starting Locust 2.43.4
--processes is not supported in Windows (except in WSL)
(.env) C:\Tests\j-l-qa-testing\Python-Locust\simple_test>
```

Single machine

It is really simple to launch a master and 4 worker processes:

```
locust --processes 4
```

You can even auto-detect the number of logical cores in your machine and launch one worker for each of them:

```
locust --processes -1
```